The Journal of Open Source Software

FisherWright.jl: forward-time Wright–Fisher simulation of whole-genome founder populations in Julia

Xijiang Yu [1¶]

1 Department of Animal and Aquacultural Sciences, Faculty of Biosciences, Norwegian University of Life Sciences (NMBU), Ås, Norway ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Julia Romanowska

Submitted: 07 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Stochastic simulations of breeding programs and complex traits require founder populations whose allele frequencies and linkage disequilibrium (LD) determine marker–QTL architecture, genomic prediction accuracy, and long-term selection response. `FisherWright.jl` generates such founders by forward-time simulation of a finite, diploid Wright–Fisher population. The model assumes a constant census size N, two sexes, random union of a sire and dam sampled with replacement, Poisson-distributed crossovers with independent assortment among chromosomes, and neutral mutation under a biallelic finite-sites model. Starting from monomorphic genomes, the simulation runs for a specified number of generations, optionally moves fixed mutations out of the active haplotypes, and exports segregating variants. Output formats include a bit-packed `BitMatrix` with a linkage map, subsampled SNP-chip panels, and native `BnGStructs.jl` haplotype containers designed for downstream breeding simulations. `FisherWright.jl` is implemented in pure Julia (Bezanson et al., 2017), MIT-licensed, and available through the Julia General Registry.

Statement of need

Simulations of genomic selection require founder genomes with site frequency spectra (SFS) and LD profiles that arise from an explicit evolutionary process. Naive Bernoulli sampling produces genotypes in linkage equilibrium, whereas heuristic approximations, such as targeting Sved's expectation $E[r^2] = 1/(1+4N_e c)$, do not reproduce the LD decay of a finite population after minor allele frequency (MAF) filtering. Coalescent simulators generate neutral equilibrium variation efficiently, but their integration into Julia workflows typically requires inter-process communication, foreign-function wrappers, or file exchange with external C and Python executables.

`FisherWright.jl` addresses this gap with an in-process, pure-Julia founder engine. It eliminates cross-language dependencies, returns the data types used by downstream breeding pipelines, and provides a transparent forward-simulation loop that can be inspected, extended, and taught without external tooling.

State of the field

Existing genomic simulation software spans several paradigms. Coalescent engines such as `msprime` (Baumdicker et al., 2022; Kelleher et al., 2016) and `MaCS` (G. K. Chen et al., 2009) are the fastest means of generating neutral equilibrium genomes; `MaCS`, for example, is the default founder generator in `AlphaSimR` (Gaynor et al., 2021). General forward-time simulators such as `SLiM` (Haller et al., 2019, 2026) and `fwdpy11` (Thornton, 2019) accommodate arbitrary selection, complex demography, and spatial structure via the Eidos scripting language or

DRAFT

The Journal of Open Source Software

Python/C++ interfaces. Specialized breeding simulators, including `QMSim` (Sargolzaei & Schenkel, 2009), `MoBPS` (Pook et al., 2020), and `XSim` (C. J. Chen et al., 2022; Cheng et al., 2015), focus on pedigree management, mating designs, and multi-generation selection programs. Among these, `XSim` version 2 is also implemented in Julia.

`FisherWright.jl` occupies a distinct niche through its deliberately limited scope. It does not implement selection, population structure, or non-stationary demography; workflows requiring those features remain better served by `SLiM` or `fwdpy11`. For neutral equilibrium founders, `msprime` is substantially faster when a Julia-only workflow is not needed. We built a new package rather than contribute to these tools for three reasons. First, the `JuliaBnG` packages needed a founder generator without non-Julia dependencies that produces the `BnGStructs.jl` haplotype type used throughout the pipeline. Second, it needed to be small enough to validate thoroughly against population-genetic theory. Third, its core algorithm originated on 5 December 2010 as a standalone C++ Wright–Fisher simulator (`pgsnp.cpp` in `pop-simulation`), which already represented haplotypes as sorted mutation coordinates, spliced parental strands across sorted crossover breakpoints, and periodically combed out fixed alleles. That code was later ported to Julia inside the author's livestock breeding simulator `xyBnG.jl` (Yu et al., 2026b), which is not publicly available, and then extracted into `FisherWright.jl` so that the founder step could be released, tested, and cited independently.

Software design

Haplotypes are represented as sorted `Vector{UInt32}` coordinates of derived alleles on a concatenated genome of up to $2^{32} - 1$ bp. This sparse representation avoids the prohibitive memory footprint of dense genotype arrays at gigabase scale. Tree-sequence recording would improve the efficiency of forward simulation, but would introduce an additional dependency and a second data model.

Meiosis is implemented as a single two-pointer traversal of the two parental haplotypes, switching strands at each crossover breakpoint (`recombine`). De novo mutations are added by a sorted merge into a preallocated buffer (`merge_sorted!`). These kernels and the crossover sampler (`cobp!`) write to preallocated thread-local buffers; regression tests verify that they perform no heap allocations after warm-up. Within each generation, meioses and mutation draws run in parallel using `Threads.@threads`. The simulation accepts an explicit random-number generator and seeds one `Xoshiro` stream per work slot before entering the threaded loops; mating is sampled serially from the supplied generator. Consequently, a realization is reproducible from the generator state independently of thread count, fixation-extraction interval, and whether fixed substitutions are returned separately. This guarantee holds within a fixed Julia version and package environment: variate algorithms can change between releases (for example, `randn` differs between Julia 1.10 and 1.13, which alters Poisson draws with mean ≥ 6 in Distributions.jl), so the same seed can yield a different realization under another Julia version.

Under neutrality, fixed derived alleles accumulate at the substitution rate μL per generation across L sites and, if retained, progressively reduce meiosis throughput. When tracking is enabled (`result = true`), a sweep at a user-defined interval (every generation by default) removes positions fixed across all $2N$ haplotypes and records them in the substitution log of a `FisherWrightResult` struct. The active haplotype vectors then contain only segregating sites, which bounds memory consumption and traversal overhead during long burn-in periods.

Data export is decoupled from forward simulation. `muts2bitarray` converts polymorphic sites to a 1-bit-per-allele `BitMatrix` with an accompanying `DataFrame` linkage map. For large genomes, where dense matrices are unwieldy, `extract_chip_bitarray` and `to_haplotype(result, positions)` evaluate a sorted subset of marker coordinates directly against sparse haplotypes to generate target SNP panels without instantiating intermediate whole-genome arrays.

The repository includes reproducible benchmark drivers to quantify the performance of this

DRAFT

design. With $N = 2{,}000$ individuals evolved for 2,000 generations ($t = N$, before equilibrium) on a 1 Gb genome (10×100 Mb chromosomes), forward simulation with v0.4.2 required 57.0 s on 8 threads of an AMD Ryzen 9 5950X (mean of three runs, range 56.9–57.1 s). Converting the resulting 571,686 segregating sites across 4,000 haplotypes to a 273 MiB `BitMatrix` required a further 6.1 s. By comparison, single-threaded `SLiM` with tree-sequence recording, which records only the genealogy during the run, required 35.0 s, and overlaying neutral mutations with `msprime` and exporting the variants required a further 6.5 s; `msprime` using its DTWF model completed the equivalent steps in 7.6 s and 5.6 s, respectively. Both tools avoid carrying alleles through each generation, which `FisherWright.jl` does by design.

Validation

The validation suite compares `FisherWright.jl` (v0.4.1) with analytical neutral expectations and matched simulations from `msprime`'s discrete-time Wright–Fisher (DTWF) model. The validation setting models $N = 100$ individuals across 10 chromosomes of 10 Mb each ($L = 100$ Mb), evolving for $20N = 2{,}000$ generations at $\mu = r = 10^{-8}$ per base pair per generation, with 50 independent replicates per simulator.

The suite comprises 24 prespecified statistical checks against theory, `msprime`, or both: crossover counts per meiosis (the mean and variance-to-mean ratio for each chromosome) and independent assortment; segregating sites (S) against Watterson's θa_n; gene diversity ($\sum 2pq$) against $\theta(1 - 1/n)$; whole-population and sampled ($n = 20$) site frequency spectra; and LD decay (σ_d^2, MAF ≥ 0.1) across five distance bins from 10 kb to 3 Mb. Expectations use $\theta = 4N_e\mu L$ with $N_e = 99.48$, the effective size of the two-sex model with binomial sex counts; `msprime`'s DTWF model has $N_e = N$, so its diversity and SFS counts are scaled by N_e/N. A check fails when $|z| > 3.5$, where z is calculated from means over the 50 replicates. All 24 checks passed in each of two independent runs (seeds 7 and 2026; maximum $|z|$ values of 2.37 and 1.91, respectively).

A lightweight statistical suite runs in continuous integration for every commit. It passes for five seeds and fails when the simulated N is halved, the recombination rate is doubled, or an earlier bug in final-generation mutation retention (inherited from the 2010 C++ loop and fixed in v0.3.5) is reintroduced. The online documentation includes the complete validation scripts, raw output logs, and a comparison with release v0.1.6.

Research impact statement

In applied breeding research with the Norwegian Association of Sheep and Goat Breeders (NSG), an early release (v0.1.x), run within the breeding simulator `xyBnG.jl` (Yu et al., 2026b), generated the historical base population used to evaluate enteric methane (CH_4) index weights and Portable Accumulation Chamber phenotyping intensities in the Norwegian White Sheep breeding program (Yu et al., 2026a). NSG will present these results to breeders at the Sheep Breeders Round Table 2026 (Jakobsen et al., 2026). That release predates the corrections listed in the package changelog; their effects on the founders are documented in the manual.

The current release is used in a pre-registered follow-up study (manuscript in preparation) that asks whether the conclusions of that evaluation depend on founder design. `FisherWright.jl` generates three of its six historical designs: the original burn-in ($N = 5{,}000$ for 5,000 generations) with the early and with the corrected release, and a small population ($N = 200$) at mutation–drift equilibrium; `msprime` generates the others. The study tests whether the corrections change the CH_4 contrast between selection scenarios, and whether that contrast, genomic prediction accuracy and the ranking of scenarios are robust to founder design.

Within the open-source `JuliaBnG` ecosystem, `FisherWright.jl` is the founder step: the breeding-scheme simulator `BnGsim.jl` (in development) and `RelationshipMatrices.jl`

DRAFT

consume its output through `BnGStructs.jl` types. The algorithm descends from a C++ program (`pgsnp.cpp`, 2010), which generated founder populations for simulation studies of genotype imputation (Yu et al., 2014), genomic prediction (Iheshiulor et al., 2016; Luan et al., 2014), and gene-editing-assisted selection (Yu et al., 2022).

AI usage disclosure

Generative AI tools were used during development and writing:

- GitHub Copilot, for code suggestions and for commits after v0.1.6; some of these list Copilot as co-author in the Git history, which was corrected in later commits;
- GPT-5, for an early code review;
- Gemini, for drafts of earlier statistical tests, which were later replaced, and for manuscript editing;
- Google Antigravity, for manuscript wording and code review;
- Claude (Anthropic), for code and manuscript review, validation and comparison scripts, documentation, and a first draft of this paper.

The author reviewed, edited, and validated all AI-assisted output and made the design decisions. Every reported number is drawn from logged runs in the repository.

Acknowledgements

The author acknowledges financial support from the Green ERA-Hub programme within the EU's Horizon Europe research and innovation programme (grant no. 101056828) and the Research Council of Norway (grant no. 351087).

References

Baumdicker, F., Bisschop, G., Goldstein, D., Gower, G., Ragsdale, A. P., Tsambos, G., Zhu, S., Eldon, B., Ellerman, E. C., Galloway, J. G., Gladstein, A. L., Gorjanc, G., Guo, B., Jeffery, B., Kretzschumar, W. W., Lohse, K., Matschiner, M., Nelson, D., Pope, N. S., … Kelleher, J. (2022). Efficient ancestry and mutation simulation with msprime 1.0. *Genetics*, *220*(3), iyab229. https://doi.org/10.1093/genetics/iyab229

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Chen, C. J., Garrick, D., Fernando, R., Karaman, E., Stricker, C., Keehan, M., & Cheng, H. (2022). XSim version 2: Simulation of modern breeding programs. *G3: Genes, Genomes, Genetics*, *12*(4), jkac032. https://doi.org/10.1093/g3journal/jkac032

Chen, G. K., Marjoram, P., & Wall, J. D. (2009). Fast and flexible simulation of DNA sequence data. *Genome Research*, *19*(1), 136–142. https://doi.org/10.1101/gr.083634.108

Cheng, H., Garrick, D. J., & Fernando, R. L. (2015). XSim: Simulation of descendants from ancestors with sequence data. *G3: Genes, Genomes, Genetics*, *5*(7), 1415–1417. https://doi.org/10.1534/g3.115.016683

Gaynor, R. C., Gorjanc, G., & Hickey, J. M. (2021). AlphaSimR: An R package for breeding program simulations. *G3: Genes, Genomes, Genetics*, *11*(2), jkaa017. https://doi.org/10.1093/g3journal/jkaa017

Haller, B. C., Galloway, J., Kelleher, J., Messer, P. W., & Ralph, P. L. (2019). Tree-sequence recording in SLiM opens new horizons for forward-time simulation of whole genomes. *Molecular Ecology Resources*, *19*(2), 552–566. https://doi.org/10.1111/1755-0998.12968

DRAFT

Haller, B. C., Ralph, P. L., & Messer, P. W. (2026). SLiM 5: Eco-evolutionary simulations across multiple chromosomes and full genomes. *Molecular Biology and Evolution*, *43*(1), msaf313. https://doi.org/10.1093/molbev/msaf313

Iheshiulor, O. O. M., Woolliams, J. A., Yu, X., Wellmann, R., & Meuwissen, T. H. E. (2016). Within- and across-breed genomic prediction using whole-genome sequence and single nucleotide polymorphism panels. *Genetics Selection Evolution*, *48*(1), 15. https://doi.org/10.1186/s12711-016-0193-1

Jakobsen, J., Cao, L., Yu, X., Difford, G. F., Berg, P., & Åby, B. A. (2026). *Integrating methane traits into sheep breeding goals*. Poster, Sheep Breeders Round Table (SBRT) 2026, Derby, UK, 20–22 November. https://nationalsheep.org.uk/events/sbrt/sheep-breeders-round-table-sbrt-2026/

Kelleher, J., Etheridge, A. M., & McVean, G. (2016). Efficient coalescent simulation and genealogical analysis for large sample sizes. *PLOS Computational Biology*, *12*(5), e1004842. https://doi.org/10.1371/journal.pcbi.1004842

Luan, T., Yu, X., Dolezal, M., Bagnato, A., & Meuwissen, T. H. E. (2014). Genomic prediction based on runs of homozygosity. *Genetics Selection Evolution*, *46*(1), 64. https://doi.org/10.1186/s12711-014-0064-6

Pook, T., Mayer, M., Geibel, J., Weigend, S., & Simianer, H. (2020). MoBPS: Modular breeding program simulator. *G3: Genes, Genomes, Genetics*, *10*(6), 1915–1918. https://doi.org/10.1534/g3.120.401193

Sargolzaei, M., & Schenkel, F. S. (2009). QMSim: A large-scale genome simulator for livestock. *Bioinformatics*, *25*(5), 680–681. https://doi.org/10.1093/bioinformatics/btp045

Thornton, K. R. (2019). Polygenic adaptation to an environmental shift: Temporal dynamics of variation under Gaussian stabilizing selection and additive effects on a single trait. *Genetics*, *213*(4), 1513–1530. https://doi.org/10.1534/genetics.119.302662

Yu, X., Cao, L., Jakobsen, J., Difford, G. F., Berg, P., & Åby, B. A. (2026a). Effects of selection weight and phenotyping intensity of enteric methane emission in national sheep breeding objectives. *Proceedings of the 13th World Congress on Genetics Applied to Livestock Production*, 23521. https://doi.org/10.31274/wcgalp.23521

Yu, X., Cao, L., Jakobsen, J., Difford, G. F., Berg, P., & Åby, B. A. (2026b). xyBnG.jl: A Julia package for scalable, GPU-accelerated stochastic simulation of complex national livestock breeding schemes. *Book of Abstracts of the 77th Annual Meeting of the European Federation of Animal Science (EAAP)*, 968. https://doi.org/10.5281/zenodo.22515787

Yu, X., Dagnachew, B. S., & Meuwissen, T. H. E. (2022). A simulator to evaluate gene editing assisted selection programs. *Proceedings of the 12th World Congress on Genetics Applied to Livestock Production*, 1442–1445. https://doi.org/10.3920/978-90-8686-940-4_344

Yu, X., Woolliams, J. A., & Meuwissen, T. H. E. (2014). Prioritizing animals for dense genotyping in order to impute missing genotypes of sparsely genotyped animals. *Genetics Selection Evolution*, *46*(1), 46. https://doi.org/10.1186/1297-9686-46-46

DRAFT